Global vs. Local Scope

Let's look at some code that breaks the rules.

```
$txt = "Let's break stuff" . PHP_EOL;

print $txt;
testScope();

function testScope(): void
{
    print $txt;
}

Output:
Let's break stuff
Warning: Undefined variable $txt
```

This code has a variable `$txt`, which is used in two places: on the first line (where it is initialized to a string), and then in the body of the `testScope()` function. On first glance this code might seem fine, but when we run it, we get a warning error that `$txt` is undefined on line 8. This happens because `$txt` isn't a valid identifier inside the body of `testScope()`. This is our first and most important rule: functions have their own scope. Variables declared outside a function are not automatically available inside a function.

When PHP executes a function, a new local scope is generated. Identifiers (and their data) created during the function's execution are isolated from the rest of the program. When execution exits a function, the scope is destroyed along with any identifiers (and data) that do not also exist outside of that scope. This is a necessary safety feature**;** otherwise, functions could inadvertently impact any other function that has the same identifiers. Imagine having to keep track of all that in your head *and* deliver features…

With this in mind, we can now see why `testScope()` is broken. As far as PHP is concerned, there is no `$txt` when it encounters it in the function body because it hasn't been defined there. Yet `$txt` does exist outside of the function, as we can see with our first call to `print`. Clearly there is another scope that exists outside of `testScope()`, and this is the *global scope*. Global scope exists for the lifetime of the program outside of any local scope. PHP differs from other languages in that global identifiers are not accessible in a local scope by default. Here's a JavaScript equivalent of the example above: